

Descrição da Solução

A nossa solução é baseada em um sistema interativo e modular de visão computacional focado na contagem e segmentação de instâncias (feijões). Buscamos permitir que o usuário possa ter controle sobre os hiperparâmetros em tempo real para conseguir lidar com variações de iluminação e sobreposição do dataset. A arquitetura foi feita com Programação Orientada a Objetos (POO) e divide-se em algumas partes principais que destacaremos abaixo.

Por conta disso, disponibilizamos três modelos diferentes, todos herdando características de uma classe base `BaseModel`, a qual faz padronização de operações morfológicas e de limiarização.

As três técnicas utilizadas foram:

- **Erosão**
- **Dilatação**
- **Watershed**

A seguir, cada técnica é descrita de forma breve.

Erosão

A erosão é uma solução morfológica direta. Ela converte a imagem para tons de cinza, aplicando limiarização (que pode ser de Otsu ou Adaptativa), o que faz os pixels encolherem com a erosão. Isso é capaz de separar feijões que estão encostados, mas sem usar a complexidade do watershed.

Dilatação

A dilatação é o oposto. Ela faz os pixels brancos "engordarem", o que ajuda a retirar manchas ou reflexos de luz das imagens.

Watershed

A técnica de watershed foca em resolver os problemas mais complexos. Ela pega uma das imagens já transformadas (erosão ou dilatação) e aplica a Transformada de Distância. Essa transformada calcula o quão longe cada pixel branco está do fundo preto, o que cria "picos" no centro de cada feijão. Ao encontrar esses picos, o algoritmo usa os mesmos como marcadores e começa a inundar a área ao redor deles, o que possibilita separar os feijões uns dos outros e poder extrair seus contornos, bem como seus centroides.

Ademais, utilizamos o OpenCV em conjunto com o Tkinter para trazer uma janela interativa, onde o usuário tem a possibilidade de definir por conta própria os parâmetros, bem como alterar a técnica em tempo real. A GUI destrói os sliders

antigos e renderiza os do novo modelo. O OpenCV garante o redimensionamento das imagens para ficarem com a mesma proporção em todas as telas.

Em conjunto com isso, temos uma função `DatasetManager` que é responsável por varrer o diretório e buscar extensões válidas, criando uma navegação circular entre as imagens do dataset.

Por fim, fizemos um arquivo JSON de configuração para cada imagem. Esse arquivo é o resultado de experimentos manuais que fizemos e nele estão as configurações que consideramos as melhores para cada imagem. Ele serve de parâmetro inicial ao carregar o dataset; desta forma, quando o usuário roda o código pela primeira vez, as imagens vêm pré-setadas com essas configurações.

Principais Etapas do Processamento

1. Carregamento e processamento da imagem

Primeiro é feita a leitura das imagens em qualquer um dos formatos mais famosos (.jpeg, .png, etc.). Em seguida é feito um downscaling; caso a imagem seja maior que 1280 pixels, o sistema reduz ela proporcionalmente.

2. Filtragem e redução de ruído

Convertemos a imagem para tons de cinza. Depois, dependendo da técnica escolhida, aplicamos ou não o `Gaussian Blur` ou o `pyrMeanShiftFiltering`, responsável por borrar as imagens e cores facilitando a identificação das bordas.

3. Limiarização

Aqui deixamos os feijões em branco e o fundo em preto. Para isso aplicamos a Limiarização Global (Otsu), onde o algoritmo analisa o histograma da imagem cinza e decide o ponto de corte ideal para separar o objeto do fundo. Também utilizamos a Limiarização Adaptativa; quando ativada, o sistema calcula o limiar de pequenos blocos da imagem, o que auxilia caso a imagem tenha muita sombra de um lado e muita luz de outro.

4. Refinamento Morfológico

Como dificilmente a limiarização deixa a imagem perfeita, aplicamos o preenchimento de buracos, onde o sistema identifica pontos de reflexão de luz dentro de feijões e pinta os mesmos de branco. Para as bordas, aplicamos erosão e dilatação: erosão para encolher e separar os feijões, enquanto dilatação faz o contrário.

5. Separação de Objetos Sobrepostos (Watershed)

Caso os objetos estejam muito próximos, a binarização pode entender tudo como um único objeto e deixar uma grande mancha branca. Para evitar isso, aplicamos a técnica de watershed, que usa a transformada de distância para calcular o pixel branco mais longe da borda preta, atribuindo o centro do feijão

ao pixel com maior valor. Após isso, o algoritmo busca os picos considerando uma distância mínima entre eles, atribuindo um marcador a cada pico. A partir dos marcadores, ele inunda os arredores até que a "água" de feijões diferentes se encontre; neste momento ele para e define os limites, traçando a fronteira exata entre os objetos.

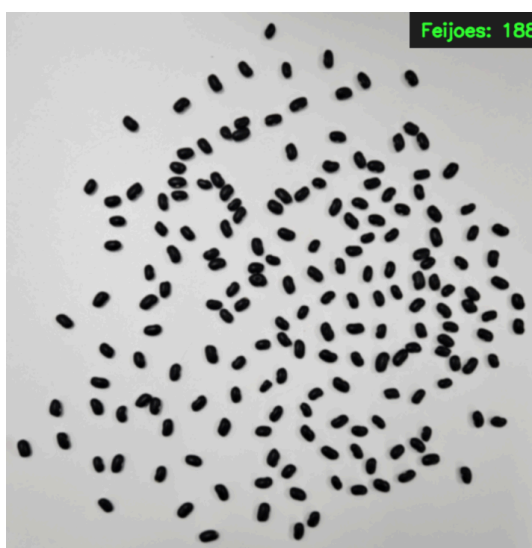
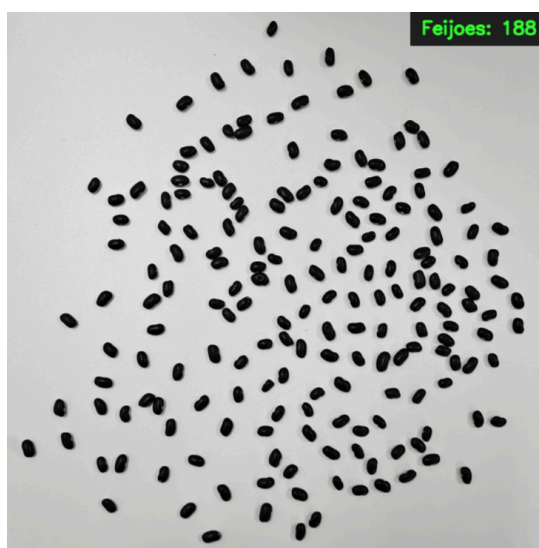
6. Extração de Características e Contagem

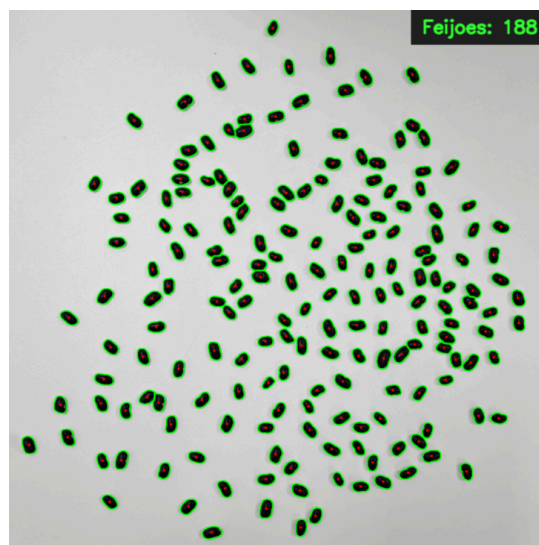
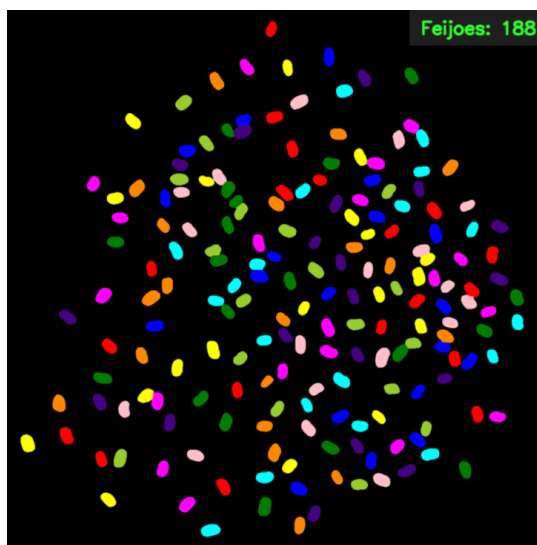
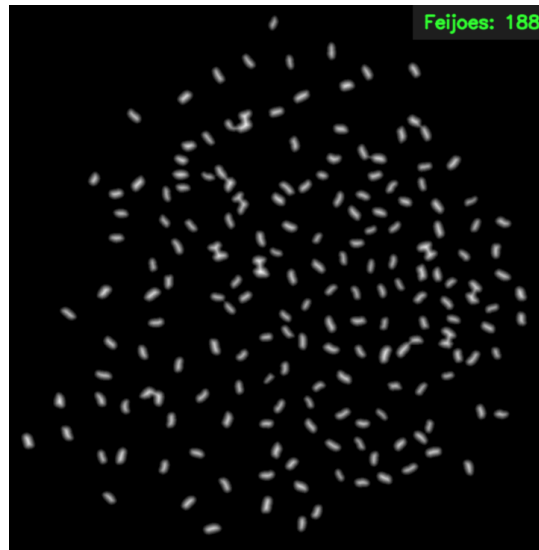
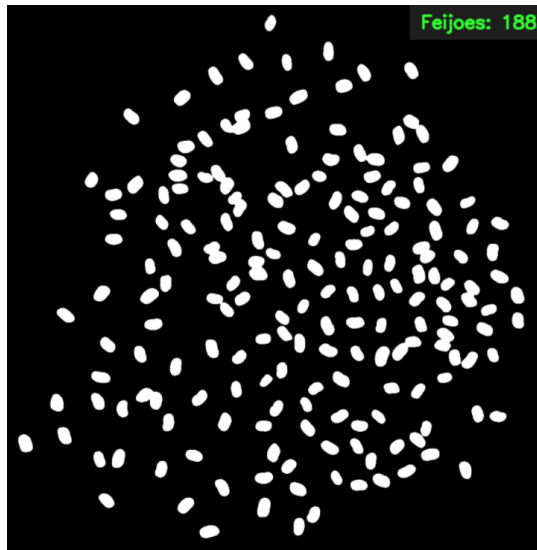
Após termos os feijões isolados pela máscara final, traçamos os contornos de cada um, buscando seu centro de massa e fazendo a contagem de todos os contornos válidos, retornando o número total de feijões encontrados em cada imagem.

7. Renderização

Por fim, temos o ViewerGUI que faz o contorno dos feijões, coloca um ponto em cada centro e adiciona no canto superior direito o valor total de feijões encontrados.

Imagens intermediárias





Resultados obtidos

Nas imagens 1 e 4 temos um cenário mais simples, com os feijões mais espalhados. Isso possibilitou o uso de erosão em conjunto com Otsu para binarizar e, aplicando um `kernel Size` pequeno (ex: 3 ou 5) com 1 iteração de erosão, soltamos os feijões com facilidade.

Nas imagens 6 e 8 ficou mais complexo, muitos feijões próximos e aglomerados, o que dificultou o processo. Foi preciso usar watershed e a transformada de distância para achar os "picos". Ajustando o parâmetro `min_distance` no Tkinter para um valor médio (entre 10 e 15), o sistema consegue identificar a maioria dos feijões com isso, porém fica um emaranhado de linhas verdes.

Por fim, no restante das imagens temos aglomerados pequenos, reflexos de luz e desafios mais medianos. Para solucionar isso, aplicamos a função `apply_hole_filling`; ela é responsável por tapar buracos brancos de forma

matemática, o que ajuda a eliminar falsos positivos da contagem, fazendo com que o feijão com reflexo valha por apenas 1 unidade.

Análise das limitações e dificuldades encontradas

Com relação às limitações e dificuldades, em imagens com feijões muito próximos uns dos outros, observamos que os algoritmos tiveram uma grande dificuldade para selecionar os indivíduos de forma separada. Mesmo com diversas parametrizações, não foi possível alcançar um resultado satisfatório nessa imagem (img6.jpg). Após pesquisas e debates, chegamos à conclusão de que o mesmo ocorreu desta forma pois a imagem apresenta feijões sobrepostos e mesmo com técnicas de erosão não foi possível separá-los a um ponto de deixar identificável pelo algoritmo.